

Performance Preset Wizard

Documentation and Quick Start Guide

1. Overview

The **Performance Preset Wizard** is an editor-only Unreal Engine plugin that helps you manage and apply performance profiles for your projects.

It introduces:

- A **data asset type** (`UPerformancePresetAsset`) to store performance profiles.
 - A dedicated **editor window** (the *Performance Preset Wizard* tab) to:
 - Browse and organize presets.
 - Apply presets with one click.
 - Configure an **Editor startup preset**.
 - A **bulk CVar editor** and **detail customization** for easier preset authoring.
-

2. Main Features

- **Performance Preset Assets**
 - Store scalability settings (View Distance, Shadows, Textures, etc.).
 - Store optional console variable overrides (CVars).
 - Include a target platform / hardware tier hint.
 - **Wizard Window**
 - Lists all presets in the project.
 - Shows a readable summary of each preset.
 - Allows applying presets with a single button.
 - **Editor Startup Preset**
 - Optionally apply a preset automatically when the editor starts.
 - **Bulk CVar Editing**
 - Human-friendly text editor for CVar overrides.
 - Import/export CVar lists as simple text.
-

3. Requirements & Installation

3.1 Engine Version

- Designed for **Unreal Engine 5.x** (tested with 5.4+).
- Editor-only plugin (no runtime dependency in packaged builds).

3.2 Installation

1. Copy the plugin folder into your project:
 - `YourProject/Plugins/PerformancePresetWizard`
 2. Open your project in Unreal Engine.
 3. Go to **Edit** → **Plugins**.
 4. Enable **Performance Preset Wizard**.
 5. Restart the editor when prompted.
-

4. Performance Preset Assets

4.1 Creating a Preset

1. In the **Content Browser**, right-click in a folder of your choice.
2. Choose **Miscellaneous** → **Data Asset**.
3. Select the **PerformancePresetAsset** class.
4. Give the new asset a descriptive name (e.g. `PP_Desktop_MidRange`).

4.2 Preset Fields

Each `UPerformancePresetAsset` contains:

- **Display Name**
 - Human-friendly name shown in the wizard.
 - Can be localized.
- **Description**
 - Short explanation of the preset (e.g. “Balanced preset for mid-range desktop GPUs targeting 60 FPS”).
- **Target Platform**
 - Enum field indicating for which hardware tier the preset is intended, e.g.:
 - *Cinematic Capture 4k*
 - *Desktop Ultra High End*
 - *Desktop Mid Range*
 - *VRHeadset PC*
 - *Laptop Development*
 - *Mobile Prototype*
 - *Custom 1–5*
- **Scalability Settings** (int32 values mapped to quality levels)
 - View Distance Quality
 - Anti-Aliasing Quality
 - Shadow Quality
 - Post Process Quality
 - Texture Quality
 - Effects Quality
 - Foliage Quality
 - These values follow the typical scalability range:
 - 0 = Low
 - 1 = Medium
 - 2 = High
 - 3 = Epic
 - 4 = Cinematic (extended)

- **CVar Overrides**
 - List of console variable overrides, each with:
 - **Name** (e.g. `r.ScreenPercentage`)
 - **Value** (e.g. `90`)
 - **bEditorOnly** (true = only in editor, false = also for runtime/standalone)
-

5. Using the Performance Preset Wizard Window

5.1 Opening the Wizard

1. In the main menu, go to:
Window → **Performance** → **Performance Preset Wizard**
The wizard opens as a **dockable tab**.

5.2 Layout Overview

The wizard consists of two main panes:

- **Left Pane – Preset List**
 - Shows all `UPerformancePresetAsset` assets sorted by:
 1. Target Platform order
 2. Display Name / Asset Name
 - Clicking an entry selects a preset and updates the details on the right.
- **Right Pane – Preset Details**
 - **Name:** Display Name or asset name.
 - **Description:** Additional info about the preset.
 - **Scalability:** Text summary of all quality levels.
 - **CVar Overrides:** Scrollable list of all CVars and their values.

At the bottom of the right pane:

- **Apply Preset** button
 - **Refresh** button
 - **Use as Editor startup preset** checkbox
-

6. Applying a Preset

1. Open the **Performance Preset Wizard** tab.
2. Select a preset from the **left list**.
3. Review its details on the **right**.
4. Click **Apply Preset**.

When a preset is applied:

- Scalability settings are converted into an `FQualityLevels` struct and passed to `Scalability::SetQualityLevels`.
- Defined CVars are applied via the global `IConsoleManager` (respecting the `bEditorOnly` flag).

This can be used to quickly switch between profiles, e.g.:

- *Desktop High End* (Epic/Cinematic)
 - *Mobile Prototype* (Low/Medium)
 - *VRHeadset PC* (high framerate settings)
-

7. Editor Startup Preset

The plugin can automatically apply a preset when the editor starts.

7.1 Configuring the Startup Preset

1. Open the **Performance Preset Wizard** tab.
2. Select the preset you want to use on editor startup.
3. Enable the checkbox:
Use as Editor startup preset.
4. The plugin saves this configuration to `UPerformancePresetWizardSettings`.

From now on:

- When the editor starts (and not running as a commandlet), the module:
 - Loads the configured preset asset.
 - Calls `PerformancePreset::ApplyPreset()` automatically.

7.2 Disabling the Startup Preset

1. Open the wizard.
 2. Either:
 - Uncheck **Use as Editor startup preset**, or
 - Choose another preset and check the box to switch.
 3. The settings are saved immediately.
-

8. Bulk Editing CVar Overrides

Preset assets provide a **detail customization** with a bulk text editor for CVars.

8.1 Opening the Detail Customization

1. Select a `UPerformancePresetAsset` in the Content Browser.
2. Open the asset in the **Details** panel.
3. In the **CVar** category you will see:
 - Buttons:
 - **Export from Array**
 - **Apply to Array**
 - A multi-line text box for bulk editing.

8.2 Exporting Current CVar Overrides

1. In the **CVar** section, click **Export from Array**.
2. The text box is filled with one line per CVar override.

Format example:

```
r.ScreenPercentage=90
r.Lumen.Reflections=0,editor
r.Fog=0
```

- Each line represents one `FPerformanceCVarOverride`.
- Optional suffix:
 - `,editor` or `,e` → `bEditorOnly = true`.

8.3 Editing and Applying CVar Overrides

1. Edit the text in the multi-line box:
 - Add new lines.
 - Change values.
 - Add or remove `,editor` suffix.
2. Click **Apply to Array**.

The plugin will:

- Clear the current `CVarOverrides` array.
- Parse each line with the pattern:
`Name = Value` or `Name = Value,editor`
- Rebuild the array using the parsed entries.
- Call `Modify()`, `PostEditChange()`, and `MarkPackageDirty()` so that:
 - Undo/Redo works correctly.
 - The asset is marked as modified.

9. Example Workflow

9.1 Creating a “Desktop Mid Range” Preset

1. Create a new `PerformancePresetAsset` (`PP_Desktop_MidRange`).
2. Set:
 - **Display Name:** `Desktop Mid Range`
 - **Description:** `Balanced preset for typical gaming desktops.`
 - **Target Platform:** `Desktop Mid Range`
3. Choose scalability:
 - View Distance = 2 (High)
 - Anti-Aliasing = 2 (High)
 - Shadows = 2 (High)
 - Post Process = 2 (High)
 - Textures = 3 (Epic)
 - Effects = 2 (High)
 - Foliage = 1 (Medium)

In **CVar** section, click **Export from Array** to initialize the text field, then add:

```
r.ScreenPercentage=90  
r.MotionBlurQuality=0  
r.Fog=0,editor
```

4. Click **Apply to Array**.

9.2 Applying and Using the Preset

1. Open **Window** → **Performance** → **Performance Preset Wizard**.
 2. Select `Desktop Mid Range` in the list.
 3. Review the details on the right.
 4. Click **Apply Preset**.
 5. (Optional) Enable **Use as Editor startup preset**.
-

10. Troubleshooting

- **The wizard window does not show any presets**
 - Make sure you have created at least one `PerformancePresetAsset`.
 - Click **Refresh** in the wizard.
 - Check that the plugin is enabled.
- **Startup preset is not applied**
 - Ensure **Use as Editor startup preset** is checked for the desired preset.
 - Confirm that you are running the editor normally (not as a commandlet or cook).
 - Verify that the preset asset still exists and was not moved/renamed without redirect.
- **CVar does not seem to have any effect**
 - Make sure the CVar name is correct.
 - Check if the project or engine overrides it elsewhere (config, command line, etc.).
 - For editor-only tweaks, use the `,editor` suffix if you do not want them in runtime.